



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目 基于智能体的漏洞挖掘与修复技术研究

学生姓名 张小北 学号 202518018629028

指导教师 侯锐 职称 研究员

学位类别 工学博士

学科专业 计算机系统结构

研究方向 漏洞治理

培养单位 中国科学院信息工程研究所

填表日期 2026 年 09 月

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	2
二 国内外本学科领域的发展现状与趋势	2
2.1 基于智能体的漏洞挖掘	2
2.2 基于智能体的漏洞自动修复	6
2.3 基于智能体的漏洞修复验证	10
三 课题主要研究内容与预期目标	17
3.1 研究目标	17
3.2 研究内容一：基于智能体的漏洞挖掘方法	18
3.3 研究内容二：基于智能体的漏洞自动修复方法	19
3.4 研究内容三：基于智能体的漏洞修复验证方法	20
3.5 预期目标与创新点	21
四 拟采用的研究方法、技术路线、实验方案及其可行性分析 ..	21
4.1 基于智能体的漏洞挖掘方法	21
4.2 基于智能体的漏洞自动修复方法	23
4.3 基于智能体的漏洞修复验证方法	25
4.4 实验方案设计	27
五 已有科研基础与所需的科研条件	28
5.1 已有科研基础	28
5.2 所需科研条件	29
六 研究工作计划与进度安排	29

一 选题的背景及意义

1.1 选题背景

安全漏洞是系统中可被威胁源利用或触发、导致安全策略被违反的缺陷或弱点，可能损害信息的保密性、完整性或系统的可用性 [1]。本课题主要关注软件与固件实现中的漏洞。与一般功能缺陷相比，漏洞分析不仅需要识别异常行为，还需要确定其触发条件与安全影响，程序是否崩溃并不是判断漏洞存在的唯一依据。

漏洞治理并不止于发现缺陷或发布补丁，而是涉及漏洞发现与报告、复现确认与根因分析、影响评估与优先级划分、修复或缓解措施制定、补丁验证、协调披露与发布，以及部署后的修复状态核查等环节 [2-4]。这些环节连接漏洞发现者、软件开发者与部署维护者，既要回答漏洞在哪里、如何修复，也要确认修复是否有效、是否体现在实际运行的程序中。因此，单个环节的自动化并不能替代对后续修复结果的确认。

近年来，大语言模型的代码理解与生成能力，为漏洞治理提供了新的技术途径。通过结合代码检索、调试、编译和测试工具，智能体能够根据执行反馈调整分析过程，逐步完成漏洞调查与程序修改。Big Sleep 已展示在真实开源软件中发现未知漏洞的能力，PatchAgent 则将缺陷定位、补丁生成与验证整合到交互式修复流程中 [5, 6]。这些进展为降低漏洞分析和修复对人工操作的依赖提供了基础，也使研究重点进一步转向复杂任务条件下的分析深度与结果可靠性。

在漏洞挖掘方面，源码可用、执行环境完备且错误判据明确时的分析能力，不能直接等同于受限场景下的发现能力。对于仅能获取设备或剥离符号二进制的嵌入式场景，程序语义恢复、内部状态观察与执行环境构建均受到限制 [7, 8]。协议状态违例、行为逻辑错误和资源泄漏等问题又可能不立即产生崩溃，需要结合协议、接口及资源生命周期要求判断行为是否错误，因而不能仅依靠增加测试输入或代码覆盖解决 [9]。

在漏洞修复方面，异常表现位置与实际需要修改的位置可能分属不同函数，局部代码和告警信息不足以完整解释漏洞根因 [10]。上下文缺失、根因理解偏差及边界条件遗漏，也可能使模型生成局部合理但修复不充分的补丁 [11]。在修复验证方面，通过已有测试并不意味着补丁在未覆盖的输入和执行路径上仍然正

确 [12]；上游补丁已经发布，也不能直接证明目标二进制包含对应修复 [4]。因此，补丁生成能力的提升还需要独立的修复有效性检查与目标产物修复状态核验相配合。

基于上述背景，本课题以“基于智能体的漏洞挖掘与修复技术研究”为题，围绕“发现问题—生成修复—确认修复”的主线，开展**基于智能体的漏洞挖掘、基于智能体的漏洞自动修复、基于智能体的漏洞修复验证**三个方向的研究，分别面向受限场景中的漏洞发现、根因驱动的补丁生成，以及修复有效性与目标产物修复状态的确认，提升漏洞治理关键环节的自动化水平与可靠性。

1.2 选题意义

在研究层面，本课题围绕安全要求与程序行为之间的对应关系，探索大模型语义推理与程序分析、动态测试相结合的方法。通过研究行为约束如何支持漏洞判定、根因理解如何指导有效修改，以及何种程序证据能够支持修复结论，有助于深化对不完备信息条件下智能体安全分析能力及其边界的认识。同时，区分漏洞发现、修复有效性与目标修复状态的评价依据，可为相关方法的设计和比较提供更明确的标准。

在应用层面，本课题有望提高嵌入式设备与二进制组件中隐蔽安全缺陷的发现能力，降低复杂漏洞定位与补丁编写的人工成本。通过同时关注安全要求与合法行为，减少不完整修复和功能回归；通过核查目标产物中的修复实现，帮助维护者识别补丁发布与实际修复状态之间的差异。相关成果可为软件安全维护、嵌入式设备审计和第三方组件更新核查提供技术支撑。

二 国内外本学科领域的发展现状与趋势

2.1 基于智能体的漏洞挖掘

2.1.1 研究背景

漏洞挖掘旨在识别目标系统中违反安全要求的程序行为，并提供支持缺陷判定与复现的证据。静态分析、符号执行和模糊测试分别从程序结构、路径约束和实际执行出发开展检测，形成相互补充的技术体系：覆盖率引导的模糊测试持续生成输入并保留有探索价值的测试用例，显著提高了自动化发现缺陷的效率；

AddressSanitizer 等运行时检测机制为越界访问、释放后使用等错误提供了可复用的判据 [13, 14]。这一体系的有效运行，还依赖目标可执行性、关键状态可达性以及错误是否具有可观察的判定依据。

大语言模型和智能体的发展正在降低代码理解、测试程序编写及分析工具编排的人工成本。Google 的 Big Sleep 已通过代码分析与工具交互，在经过长期测试的 SQLite 中发现未知内存安全漏洞；Mozilla 于 2026 年 4 月报告，使用前沿模型在初步评估中发现的 271 项漏洞已随 Firefox 150 修复 [5, 15]。这些进展说明智能体已能参与成熟项目的实际安全分析，也表明内存安全问题尚未被普遍解决；在长期充分测试的特定组件中，仅继续增加同类输入变异的边际收益正在下降，新的增量能力更加依赖对深层程序语义与测试盲区的理解。因此，后续研究需要考察自动化能力尚未充分覆盖的任务条件。

相比能够通过崩溃、异常或崩溃报告直接暴露的内存错误，许多安全缺陷并不存在统一、显式的错误信号。例如，协议实现可能违反状态转移要求，权限状态可能在错误的会话间延续，资源也可能在生命周期结束后持续占用，而程序仍能正常执行。此类问题的关键不再只是能否触发新的执行路径，而是能否获得相应的安全要求，并据此判断实际行为是否构成违例，即测试预言（test oracle）问题 [9]。因此，本文重点关注协议与状态语义、访问控制和资源生命周期等需要结合行为约束才能判定的安全缺陷。

上述问题在嵌入式与固件场景中进一步受到分析条件的限制：仅能与设备交互时，内部状态、代码覆盖及资源对象通常不可直接观察；剥离符号的二进制需要恢复对象、功能与程序位置之间的对应关系；固件重托管后的行为也可能依赖外设响应、配置、定时事件与跨组件交互。Firmalice 和 Fuzzware 分别从二进制安全语义分析与硬件接口建模出发处理其中的部分困难，说明这些限制需要专门的程序分析与执行支持 [8, 16]。

因此，本课题关注在**源码、符号、规约或运行时观察不完备的条件下，如何利用大语言模型的语义理解与交互分析能力，发现违反行为约束且能够在真实环境中成立的安全缺陷**。研究重点由单纯增加输入与路径覆盖，转向安全要求的获得、目标行为的辨识、状态违例的触发及其可信判定。

2.1.2 研究现状与趋势

传统漏洞挖掘主要依赖静态分析、符号执行和模糊测试，并逐渐从代码路径覆盖扩展到程序状态和安全性质。AFLNet、StateAFL 等工作利用协议响应或运行时状态反馈增强有状态程序的路径探索 [17, 18]；LTL-Fuzzer 等进一步将显式行为性质作为测试目标，搜索违反时序约束的执行轨迹 [19]。这些方法提高了复杂状态空间中的探索能力，但通常需要预先定义状态表示、安全性质及其程序事件映射。对于缺少完整规约和语义信息的目标，如何获得可信的安全要求并将其转化为可执行的测试判据仍是重要问题。

大模型的发展进一步增强了漏洞挖掘中的输入生成和分析能力。Fuzz4All 利用 (Large Language Model, LLM) 生成和变异测试输入，ChatAFL 则利用模型的协议知识生成消息并探索新的协议状态 [20, 21]。近期研究进一步将模型置于持续的工具交互过程中，使其参与代码理解、假设生成、测试和证据分析。相比传统模糊测试，这类方法显著增强了输入构造和分析过程的自动化，但“能够探索更多行为”并不意味着“能够正确判断安全违例”，安全假设和测试 oracle 的可靠性仍需独立解决。

这一问题在缺少源码和符号信息的二进制与固件场景中更加突出。BINREX 结合语义检索、分层规划和 IDAPython 工具调用实现任务驱动的二进制分析 [7]；FirmAgent 则结合模糊测试与 LLM 污点分析，在固件中建立实际输入与潜在危险路径之间的联系 [22]。与此同时，Fuzzware 等工作通过建模 MMIO 等外部环境提高固件动态执行能力 [8]。这些研究改善了语义恢复、路径定位和固件可执行性，但对于依赖历史状态、环境变化和多步交互的漏洞，仅恢复代码语义或提高执行覆盖仍不足以形成可靠的安全判据。

因此，近期研究开始进一步利用协议规范和历史缺陷知识提供安全语义。BSFuzzer 利用大语言模型从蓝牙协议规范中恢复状态机和报文语义，并据此构造字段和状态转移违例；ProtocolGuard 从协议规范提取约束，将其映射到源码并生成可执行断言进行验证 [23, 24]。另一类工作则从历史修复中提取可复用的行为约束，例如 TyPatch 利用 LLM 从 Linux 补丁中提取对象、动作、守卫和状态转移等类型状态转移规则 [25]。这些工作说明，规约和历史补丁都可以为漏洞挖掘提供比代码覆盖更明确的安全要求，但现有方法通常依赖较完整的规范、源码或静态分析条件，如何在信息受限的二进制和设备环境中恢复并验证这些约

束仍有困难。

此外,大量逻辑和资源缺陷并不会直接产生崩溃,使漏洞挖掘进一步面临“如何判错”的问题。LISA 通过生成 API 调用序列和程序不变式发现功能异常 [26], InferROI 则利用 LLM 推断资源获取、释放和可达性意图,并结合静态分析检测资源泄漏 [27]。这些工作表明,漏洞检测可以利用崩溃之外的行为和资源性质,但对于长期资源占用、状态错误和协议逻辑异常,仅观察执行结果、资源曲线或对象可达性通常不足以判断是否真正违反安全要求,还需要结合对象生命周期、历史状态和预期行为进行解释。

综合来看,漏洞挖掘正在由单纯追求输入和路径覆盖,逐步转向基于状态、规约和安全性质理解程序行为;大模型与智能体的发展又进一步增强了语义恢复、工具使用和测试生成能力。然而,现有能力仍主要分散在状态探索、规约提取、二进制分析、环境建模和定制 oracle 等不同环节。进一步需要解决的核心问题是:在源码、规约和执行环境均可能不完整的条件下,如何获得可信的安全要求,将其映射为目标程序可观察、可执行的行为条件,并形成真实可达、可复现的漏洞证据。

2.1.3 研究挑战

首先,行为规约的可信获得是语义漏洞挖掘的基础。协议规范、源码和历史补丁分别描述预期行为、实际实现和历史修复条件,其完整性、适用范围和可信度并不一致。已有研究表明,规约解释或上下文缺失会直接影响后续检测结果 [24, 25]。因此,在缺少完整规范时,需要区分明确规定的安全义务、具有独立证据支持的推断和待验证假设,避免直接将程序中的常见行为或模型生成的解释作为安全规范。

其次,需要解决安全概念与目标程序行为之间的映射。在缺少源码和符号信息的场景中,即使能够恢复函数语义,也仍需确定安全要求具体对应哪个对象、状态和程序位置,以及状态变化如何影响后续行为。BINREX 等工作增强了剥离二进制中的语义恢复能力 [7],但对于跨函数、跨任务和多组件共同维护的状态,仅识别函数功能仍不足以判断安全约束是否被违反。设备黑盒场景下内部状态进一步不可见,同一响应也可能对应不同内部状态,因此需要根据源码、二进制和设备等不同可观察条件建立不同层次的行为证据。

进一步地，状态性漏洞要求构造**真实可达的违例轨迹**。协议状态、资源生命周期和硬件环境通常由多次调用及外部事件共同决定，已有状态感知模糊测试能够增强多步交互探索 [18, 21]，但生成任意非法操作往往只会触发早期拒绝，并不能到达真正需要检查的状态。在重托管和模拟执行环境中，还必须区分人为构造的内部状态与攻击者能够通过合法外部操作到达的状态。只有能够说明目标状态的到达过程及其环境假设，异常行为才能形成具有实际意义的漏洞证据。

另一个核心问题是**缺少直接错误信号时的语义判定**。逻辑错误、协议违例和资源泄漏通常不会产生崩溃，需要通过不变式、断言或定制 oracle 判断行为是否违反安全要求 [24, 27]。这类判据既不能过弱而遗漏漏洞，也不能过强而拒绝合法行为；资源类缺陷还涉及生命周期和观察窗口，例如未立即释放可能来自异步回收，资源增长也可能来自合法缓存。因此，需要同时定义预期安全行为和合法对照行为，并将候选发现与独立验证分离，使最终结论能够被重复执行和反驳。

最后，智能体化漏洞挖掘还需要从单点能力提升转向**完整发现过程的可靠评价**。代码覆盖、状态覆盖、性质监测状态和资源消耗都只能反映分析过程的部分信息，代理指标的提升并不必然意味着发现了更多真实漏洞。评价时还需控制源码、规约、历史补丁等输入条件，区分静态候选、可触发缺陷和经确认漏洞，并记录工具执行、环境构建和人工确认等成本。只有保持输入条件和证据标准一致，才能判断性能提升究竟来自模型能力、分析方法还是额外信息。

综上，智能体与大模型增强了语义理解、状态探索和工具使用能力，但并未消除规约缺失、状态不可见和非崩溃行为难以判定等根本问题。面向嵌入式设备和剥离二进制，本课题需要进一步建立**可信行为约束、真实可达交互与可靠语义判定之间的联系**，并以此连接行为规约、状态探索、可达性分析和漏洞判定。

2.2 基于智能体的漏洞自动修复

2.2.1 研究背景

漏洞修复是软件安全维护的重要环节，其目标除阻止已知攻击输入触发异常外，还需消除导致危险行为的程序缺陷，并维持正常使用所依赖的功能与安全约定。漏洞自动修复（Automated Vulnerability Repair, AVR）将漏洞分析、补丁生成与补丁验证组织为自动化过程，以降低安全维护对人工分析的依赖。与面向一般软件缺陷的自动程序修复（Automated Program Repair, APR）相比，AVR

更强调在攻击者可控输入、异常状态及安全敏感操作下恢复必要的安全条件。修复目标需要同时考察危险行为是否被有效约束、相关合法行为是否得到保留，代码修改是否合理、程序能否编译或已知失败是否消失只能作为局部指标 [3]。

漏洞修复通常发生在不完备信息条件下。漏洞报告、检测器告警和概念验证程序（Proof of Concept, PoC）能够提供分析线索，但不一定直接指出应修改的程序位置，也不一定完整描述正确修复应满足的条件。在漏洞修复中，已知 PoC 不再触发，可能只是某个特定输入或执行分支被阻断，漏洞根因是否消除仍需独立验证。由此，AVR 实际上包含两个相互关联的问题：**如何根据有限证据推断应当修复什么，以及如何验证生成的补丁确实满足修复要求** [3, 12]。

LLM 为处理漏洞描述、源代码和修复经验提供了新的技术途径。进一步结合代码检索、文件编辑、编译执行与测试工具，模型能够在智能体框架中逐步完成定位、修改和反馈驱动的修复。PatchAgent 等工作已经将这些环节整合到交互式系统中，推动研究对象从给定代码片段上的补丁生成扩展至真实项目中的自动修复 [6]。交互能力的增强提出了更高的语义理解要求：系统仍需判断告警位置与根因位置之间的关系、识别跨函数依赖，并区分补丁通过当前验证与恢复完整安全要求之间的差别。

Rust 等系统编程语言使这一问题具有更丰富的语义层次。RustBelt 从形式化角度说明，基于不安全 Rust 代码（unsafe Rust code）实现的安全抽象需要满足相应验证条件，才能作为安全语言的可靠扩展 [28]；针对真实 Rust 程序的研究也揭示了内存与线程安全实践中的复杂问题 [29]。因此，Rust 漏洞修复不仅涉及局部条件检查，还可能涉及所有权、生命周期、类型与 trait 约束、异常清理和并发状态。基于上述背景，本节围绕传统修复方法、LLM 与智能体修复、补丁正确性评价及 Rust 修复进展展开，重点讨论从漏洞表现恢复根因语义、从候选修改建立可信修复证据所面临的挑战。

2.2.2 研究现状与趋势

传统自动程序修复（Automated Program Repair, APR）已经形成了基于搜索、程序合成和语义约束等多种技术路线。GenProg 通过搜索程序变换空间寻找能够通过测试的候选补丁，SemFix 则利用符号执行和程序合成求解满足测试约束的修复表达式 [30, 31]。面向安全漏洞，ExtractFix 等工作进一步从漏洞执行中提取

安全约束，并将其传播至候选修复位置以指导补丁生成 [32]。这类方法将修复从简单的测试通过推进到显式约束满足，但其能力通常受漏洞模型、程序分析精度和候选修复空间限制，对复杂跨过程语义和缺少明确错误信号的安全问题仍难以建立完整的修复条件。

大语言模型的发展显著扩展了漏洞修复中的候选补丁生成能力。VulRepair 等工作从历史漏洞补丁中学习漏洞代码到修复代码的映射；且零样本漏洞修复研究表明，通用代码模型无需针对特定漏洞类型训练，也能够根据漏洞代码和提示信息生成有效候选补丁 [33, 34]。此后研究进一步引入漏洞上下文和根因信息，例如 APPATCH 结合静态依赖分析、根因推理和历史修复经验组织模型所需的代码上下文 [11]。这些工作表明，LLM 修复正在由局部代码生成转向利用跨函数上下文和漏洞语义进行修复，但模型生成的补丁是否真正恢复安全性质仍需要程序级验证。

进一步地，修复系统开始从固定的生成流程发展为能够自主操作真实仓库的智能体。SWE-agent 展示了模型通过代码导航、文件编辑和程序执行完成复杂软件工程任务的能力，PatchAgent 等工作进一步将漏洞定位、补丁生成和测试反馈纳入持续的交互式修复过程 [6, 35]。这类方法使模型能够根据构建、漏洞复现和回归测试结果不断调整补丁，但多轮迭代也可能使补丁逐渐适配已有测试，而没有真正消除漏洞根因。因此，智能体自主性的提高并不能替代对补丁安全性的独立验证，修复位置、根因解释和最终安全性质需要分别考察。

与修复能力的发展相对应，真实漏洞基准逐渐从提供漏洞代码转向提供可复现的仓库级任务。Vul4J 为 Java 漏洞提供可执行的真实项目和漏洞证明测试，SEC-bench 则将类似任务扩展到真实 C/C++ 仓库，并探索自动化构建漏洞修复案例 [36, 37]。这些基准为端到端修复提供了实验基础，但仅依赖已有 PoC 和功能测试仍面临经典的测试过拟合问题：补丁可以使已知触发失效并通过回归测试，却仍然没有恢复漏洞所要求的安全性质。

因此，近期研究开始加强补丁的独立安全验证。PVBench 利用官方修复新增的测试检查基础验证遗漏的漏洞相关行为，PatchBench 则通过额外漏洞输入和正常行为对照进一步检查补丁的安全性与功能保持 [38, 39]。这些工作表明，已知 PoC 消失并不足以证明漏洞被正确修复；同时，评估阶段能够使用的官方补丁和隐藏测试通常不应直接暴露给修复系统。因此，**修复过程能够获得的信息与**

独立评估所使用的安全证据需要明确隔离，并进一步区分漏洞必要的安全要求与官方实现中的其他工程选择。

在这一方向上，本课题组前期构建了面向 Rust 真实漏洞的仓库级修复基准 **RRBench** [40]，包含 115 个可复现漏洞案例。RRBench 将补丁评价划分为公开验证和私有验证两个阶段：公开阶段提供漏洞触发 PoC 与功能测试，私有阶段利用修复系统不可见的安全性质和合法行为检查验证补丁是否真正恢复漏洞要求。对代表性修复智能体的实验表明，通过公开验证的补丁中仍有 23.4% 无法通过私有验证，并有 14 个案例未被任何配置正确修复。该结果说明，**端到端漏洞修复的关键问题已经不仅是生成能够通过测试的补丁，还包括如何理解漏洞根因、形成正确修复，并利用独立安全性质验证其正确性**，为后续研究“生成—验证”协同的漏洞修复方法提供了直接基础。

2.2.3 研究挑战

2.2.4 研究挑战

首先，复杂漏洞修复需要从不完备的定位信息中恢复**根因位置与必要上下文**。已有研究表明，漏洞触发位置与实际修复位置可能跨越不同函数，根因分析需要结合静态依赖和动态证据 [10, 11]。当上游仅提供崩溃位置、污点终点或局部告警时，系统仍需追踪状态的建立、传播和使用关系；同时，上下文过少可能遗漏关键依赖，过多又会引入无关信息。因此，关键问题是如何将初始定位线索转化为可验证的程序依赖证据，并据此动态确定分析和修复范围。

在此基础上，还需要保证**根因判断与修复策略之间的一致性**。漏洞类别和历史案例能够提供修复经验，但并不能直接确定目标程序中的缺陷对象、形成原因和正确干预位置。现有 LLM 修复方法已经利用根因推理和历史经验辅助补丁生成 [11, 41]，但错误的早期解释也可能持续影响上下文选择和候选生成。因此，根因假设需要接受控制流、数据流、接口约束和执行证据的检验，并在证据冲突时重新定位或调整修复策略，而不是围绕单一假设反复生成相似补丁。

进一步地，正确修复要求恢复**跨接口和状态的安全性质**。真实漏洞的安全要求可能由多个函数、对象状态和调用阶段共同维护，尤其在 Rust 等具有丰富类型和生命周期机制的系统语言中，漏洞表现位置并不一定是安全约束应当建立的位置 [28, 29]。修复不仅需要消除已知触发，还应确定安全性质由哪些操作共

同维护以及是否存在绕过路径。对于部分漏洞，增强类型或接口约束也可能构成有效修复，因此评价时还需区分对危险行为的有意限制与对合法程序行为的意外破坏。

由此产生的核心问题是**如何将安全性质转化为独立、可执行的补丁验证**。已有研究通过额外漏洞输入、开发者新增测试和行为对照增强补丁验证 [38, 39]，但有限测试仍可能接受不完整修复，过强的检查又可能拒绝不同于官方实现的正确方案。因此，需要从漏洞根因中提炼必要的安全要求，将其转化为可执行检查，并同时设置合法行为对照。修复阶段可以利用公开测试和生成的反例持续迭代，但最终验证应与修复过程保持信息隔离，避免修复器针对隐藏判据过拟合。有限验证只能证明补丁满足已检查的安全要求，而不能排除未覆盖执行空间中的残余风险。

最后，智能体修复需要建立**统一且可复现的端到端评价标准**。现有研究在是否提供漏洞位置、是否要求仓库级分析以及采用测试、参考补丁还是人工审查等方面存在明显差异 [37, 39]。评价时应明确系统能够获得的信息，分别记录补丁生成、构建、公开验证和独立安全验证结果，并保留无补丁、环境失败和预算耗尽等情况。同时还需要控制历史补丁泄漏以及模型、工具链和执行环境差异，并将上下文分析、候选生成和验证成本纳入评价，从而区分模型能力、额外信息和分析机制各自带来的收益。

综上，智能体漏洞修复已经从局部代码生成发展到真实仓库中的交互式分析，但可信修复仍依赖两个相互衔接的核心能力：**一是不完备信息中形成有证据支持的根因判断并生成相应修复，二是将漏洞安全要求转化为独立、可执行且兼顾合法行为的验证条件**。本课题研究内容二围绕这一“生成-验证”协同过程展开，研究根因分析、候选修复与安全性质验证之间的联系，以提高复杂真实漏洞自动修复的可靠性。

2.3 基于智能体的漏洞修复验证

2.3.1 研究背景

漏洞修复是软件安全保障的重要环节，其目标除消除已知漏洞的触发条件外，还需要在维持合法功能的前提下阻断危险行为。围绕这一目标，自动漏洞修复（Automated Vulnerability Repair, AVR）研究形成了漏洞分析、补丁生成与补

验证三个主要环节，采用程序分析、约束求解、程序变换和学习等技术降低对人工修复的依赖 [3]。随着大语言模型的发展，修复过程进一步由单一补丁生成扩展为交互式任务执行，PatchAgent 将缺陷定位、补丁生成与验证组织到同一智能体中，利用语言服务器和验证工具支持多轮修复 [6]。这些研究主要回答“如何产生有效修复”，软件系统的实际安全状态还取决于修复能否被正确集成并传播至最终部署的程序。

从软件供应链的角度看，漏洞修复远超上游提交补丁这一局部开发活动，它贯穿上游项目、下游分支、发行版、设备厂商与最终用户，构成一个持续维护过程。Li 和 Paxson 对安全补丁的研究表明，补丁开发需要同时考虑修复完整性与功能回归，安全修复也可能仅涉及少量代码变化 [42]。针对修复在相关仓库之间的传播，Machiry 等提出 SPIDER，通过分析补丁的行为影响，识别适合向相关项目传播的修复 [43]；Zhang 等对 Android 内核补丁生态的研究进一步发现，修复在多层供应链中的传播可能存在较长延迟 [44]。因此，“补丁已经公开”“下游已经合入”与“部署产物实际包含修复”是三个需要分别确认的状态。

版本信息和更新公告能够辅助判断修复状态，其覆盖范围存在边界。一方面，下游维护者可能将安全修复回移植至较早版本，同时保留原有上游版本号，Red Hat 的安全回移植实践明确指出，仅依据上游版本号开展漏洞扫描可能产生误判 [45]。另一方面，闭源产品、固件和第三方二进制组件往往无法提供与部署产物完全对应的源码，使用者难以直接检查安全修复是否进入最终程序。由此，针对实际二进制开展独立的修复状态核查，成为连接补丁发布与安全维护的重要技术需求 [4]。

补丁存在性测试（Patch Presence Testing，PPT）正是针对这一需求提出的：给定已知漏洞的修复信息，以及补丁状态未知的目标程序，判断目标中是否包含对应修复的实现证据。不同方法使用的参考材料有所区别，包括源码补丁、修复前后源码、参考二进制或中间表示等 [4, 46]。PPT 与通用二进制相似性检测的区别在于，后者主要寻找功能或结构相近的代码，前者则需要在高度相似的两个实现之间识别安全相关差异：一个边界条件、错误分支或参数处理的变化，可能不足以明显改变函数整体相似度，却足以改变其漏洞状态 [46]。

需要区分的是，**PPT 核查“特定修复是否体现在目标程序中”**，**补丁正确性验证考察“修复是否充分消除目标缺陷并维持合法行为”**。即使检测到参考修复

已经存在,也不能据此证明参考补丁本身完整,更不能推导出目标程序不存在其他漏洞 [3, 4, 42]。因此,本节将 PPT 定位为漏洞修复生命周期中的部署侧验证环节,重点讨论其二进制证据提取、补丁语义匹配以及智能体化分析能力。

2.3.2 研究现状与趋势

(一) 基于局部语法与结构差异的补丁存在性测试 基于语法与结构差异的方法通过比较修复前后的程序变化构造补丁特征,再在目标中搜索相应特征。与直接比较整个函数相比,这类方法将分析范围收缩到补丁影响区域,以减少无关代码变化对判定的干扰 [46, 47]。

Xu 等提出的 BinXray 采用两阶段匹配:先通过函数签名检索疑似目标函数,再通过修复前后函数之间的基本块映射提取补丁签名,利用长度敏感的相似性比较区分漏洞版本与修复版本 [46]。其重要特点是分析可以在二进制层面完成,不要求提供目标源码,并将函数检索与补丁匹配纳入同一流程。由此可见,传统方法已经开始关注目标定位,函数级相似性只能提供候选范围,补丁级证据的识别仍需进一步研究。

PatchDiscovery 进一步关注补丁签名的选择性:在规范化和简化的控制流图上匹配基本块,分别提取漏洞版本与修复版本的关键基本块,再在目标函数中比较其匹配情况 [47]。关键基本块减少了无关指令和版本变化引入的噪声。这一路线的共同优势是特征直观、匹配流程相对明确,适合快速筛查;局部结构特征仍需要在参考程序与目标程序之间保持可对应性,编译优化、基本块重组以及补丁之外的结构变化,都会使“同一修复”与“相似结构”之间的对应关系更加复杂 [46, 47]。

(二) 基于符号执行与中间表示的语义分析 为降低对指令形式的依赖,研究者开始利用符号执行和中间表示描述补丁引起的行为变化。Zhang 和 Qian 提出的 FIBER 结合源码变更分析与二进制符号执行,将选定的补丁位置转换为可匹配的二进制签名,并在匹配时结合局部控制流结构与语义公式 [4],通过结构约束与语义约束的结合,增强签名对二进制表示差异的适应能力。FIBER 的实际评估使用带有符号信息的内核镜像;对于符号不可用的情形,论文讨论了接入相似性检索进行候选函数定位,但未在该工作中完成这一集成。

Jiang 等提出的 PDiff 面向经过厂商定制的下流内核，通过锚点基本块限定补丁影响路径，再利用路径约束、内存状态和函数调用等信息生成语义摘要，比较目标与修复前后参考版本的接近程度 [48]。该设计将比较对象从表面指令扩展为补丁相关路径上的行为，能够减轻定制代码的干扰；其目标函数定位使用内核中的 KALLSYMS 信息，路径枚举也需要采取循环展开等近似策略，仍存在输入条件和分析范围上的边界。

Yang 等提出的 Robin 则通过符号执行寻找通向补丁区域的路径，构造能够区分修复前后行为的函数输入，并比较相同输入下的执行特征 [49]，强调使用补丁引起的行为差异校验相似函数。与此同时，函数级欠约束执行中得到的路径和输入，与完整程序中的真实可达执行之间仍有差距，“行为特征相似”补丁存在”与”漏洞可实际触发”需要分别讨论。

PS³ 通过符号模拟提取寄存器写入、内存存储、条件与函数调用等语义签名，利用参考与目标之间的签名比较降低编译选项变化的影响 [50]；REACT 将源码编译得到的 LLVM IR 与目标二进制提升得到的 IR 纳入统一分析流程，通过语义特征提取、特征细化与排序，以及约束求解进行补丁判定 [51]。这些方法拓展了 PPT 的语义比较能力，分析精度仍取决于符号模拟、二进制提升和特征抽象的准确性：IR 恢复误差与不完整的执行模型可能影响后续匹配，即使采用约束求解，比较结论仍受所提取特征与建模条件的限制。

(三) 不同程序表示下的补丁检测与演化问题 除原生二进制外，Java 和 Android 字节码也形成了相应研究路线。BScout 直接检查 Java 可执行文件中的完整补丁，突破了仅依赖少量局部签名的传统做法 [52]；PPT4J 结合补丁的语义信息与字节码特征，建立源码补丁行和二进制实现之间的对应关系 [53]；PHunter 面向 Android 应用中的混淆代码，通过由粗到细的定位与匹配提升补丁识别能力 [54]。这些方法表明，分析对象保留的信息、代码变换方式以及定位粒度，会直接影响技术设计；字节码场景的能力向经过优化和符号剥离的 C/C++ 原生二进制推广时，需要重新评估信息条件。

另一个需要独立考察的问题是安全补丁演化。Xie 等系统研究了补丁代码在后续软件版本中的持续变化，以及这些变化对已知漏洞分析工具的影响 [55]。编译差异主要改变同一源码实现的机器级表达，补丁演化则可能直接改变修复实

现，例如重组检查逻辑或调整相关代码；能够抵抗编译选项变化，距离识别经过长期重构的修复仍有差距。由此可见，传统方法的关键问题除生成更加稳定的签名外，还包括厘清签名所代表的安全行为，以及该行为在不同实现中成立所需的条件。

(四) 基于大语言模型的补丁语义分析 大语言模型为连接高层修复信息与低层程序表示提供了新的技术途径。DeGPT 利用模型改善反编译结果的可理解性，LLM4Decompile 探索从二进制代码生成或改进类源码表示，ReSym 则结合模型与程序分析恢复剥离二进制中的变量和数据结构符号 [56–58]。这些工作提供的是程序理解的辅助能力，并未直接回答特定安全补丁是否存在；其价值在于为跨表示的语义定位与比较提供了基础。

Li 等提出的 Lares 是直接面向 PPT 的 LLM 方法：从修复前后源码提取补丁相关切片，补充数据依赖、控制依赖和宏信息，再由 LLM 将切片映射到目标函数的反编译伪代码，随后利用 Z3 检查可提取公式的等价性，并在不能直接判定时使用 LLM 比较语义 [59]。这种设计消除了为提取参考特征而重新编译历史版本的要求，将模型用于跨表示定位，突破了仅作为最终二分类器的用法。

Lares 的适用边界同样重要：**其方法明确假设潜在漏洞函数已由外部二进制相似性方法识别，所解决的定位主要限于目标函数内部的代码切片定位，尚未覆盖从完整二进制独立完成全部检索步骤的场景。**其失败分析还指出，重复检查与细微修改可能干扰判断 [59]。”能够处理剥离后二进制的伪代码”代表局部切片定位能力，端到端分析还需要独立完成检索步骤；局部公式匹配与完整安全语义验证之间也需要补足中间环节。

Xu 等提出的 PPTSP 采用另一种混合设计：先对语义等价的指令序列进行规范化，再提取补丁影响的关键控制路径以减少无关代码干扰，最后在结构特征相近时引入 LLM 判断语义等价性 [60]。Lares 将模型用于源码—伪代码映射，PPTSP 主要将模型用于传统特征处理之后的语义判别，两者体现出互补的结合方式：模型可以辅助定位候选证据，也可以处理已有特征难以区分的情况，不同使用位置带来的收益与风险需要分别评估。

(五) 从 LLM 增强流水线到交互式智能体分析 将 LLM 加入某个分析阶段，与由智能体组织完整分析过程属于两个层次。SWE-agent 的研究表明，专门设计的

智能体—计算机接口能够支持模型自主浏览仓库、执行程序并根据反馈调整操作 [35]。结合这一交互范式，可以进一步探索由智能体围绕补丁假设主动选择二进制分析工具，在候选检索、局部检查与证据补充之间迭代，避免始终沿固定顺序运行预定义模块。通用代码任务与 PPT 在分析对象和证据要求上存在差异，这种交互能力能否转化为可靠的端到端补丁检测，仍需要专门研究。

这一变化还意味着评价对象需要从单次模型输出扩展为完整分析系统。模型、工具接口、上下文组织、停止条件与重试机制都会影响执行过程；有关智能体评价的研究也指出，仅关注任务准确率而忽视资源使用和复现条件，可能对系统实用性产生误判 [35, 61]。对 PPT 而言，需要进一步回答：模型能否自行找到正确代码，工具返回的信息能否构成补丁证据，失败发生在哪个环节，以及正确结论需要付出多少分析成本。因而，现有研究呈现出的发展方向已超越简单地“以 LLM 替代传统程序分析”，更强调将程序分析提供的可检查证据与模型的语义推理、任务规划能力结合，并对整条流程进行验证。

围绕上述问题，本人前期已开展两项相互衔接的实证研究。其一，针对现有 PPT 研究评估数据集有限且不一致、失败根因缺乏系统理解的问题，构建了包含 561 个 CVE、覆盖 10 个广泛使用开源项目的高保真基准，对五款代表性工具进行系统评估，揭示了“高准确率、低决策成功率”的普遍现象，并将失败模式归纳为算法层局限与工程层缺陷两大类 [62]。其二，针对智能体能否胜任端到端 PPT 这一问题，系统评估了通用编码智能体在符号可见、剥离二进制、演化补丁与真实发行版包等场景下的能力边界，发现前沿智能体仅使用 binutils 即可达到与专用 SOTA 系统相当的性能，但补丁定位与演化补丁识别仍是主要瓶颈 [63]。这两项工作作为研究内容三的提出提供了直接的实证基础。

2.3.3 研究挑战

(一) 缺少定位提示时，补丁语义与目标代码之间的可靠对应 现有方法在定位方面具有不同前提：FIBER 和 PDiff 的评估或实现利用目标符号信息，BinXray 包含函数级检索，Lares 则假设目标函数已由外部方法识别 [4, 46, 48, 59]。这些差异意味着，已知目标函数上的匹配效果不能直接代表完整二进制上的端到端效果。面向实际部署，分析不仅需要找到语义相关的函数，还必须确定具体检查作用于哪个对象、处于哪个程序位置，以及是否约束了对应危险操作。后续研究

需要将函数检索、程序点定位与语义验证联系起来，并在候选存在歧义时继续收集证据，避免直接接受首个相似结果。

(二) 参考实现变化时，修复语义的保持与可观察性 语法签名、符号执行摘要和 IR 特征分别缓解了不同形式的程序差异，但它们仍基于一定的实现对应关系 [46–51]。补丁演化研究则表明，原始修复代码会在后续版本中持续变化 [55]。因此，需要进一步区分“原有补丁模式不再出现”与“修复效果不再存在”：前者可能只是实现调整，后者才涉及修复缺失。针对这一问题，应围绕检查对象、必要前提、错误传播和危险操作之间的关系组织验证，并按需扩展至调用上下文，避免仅停留在提高局部签名的匹配容忍度。同时，当给定分析范围内缺乏可区分证据时，应保留判断边界，模型推测需要明确标注为待验证假设。

(三) 模型推理与程序证据之间的一致性 Lares 和 PPTSP 将模型用于定位或语义判别，说明自然语言推理能够参与传统分析难以直接处理的环节 [59, 60]。模型生成的解释、反编译输出以及求解器结论具有不同的证据性质：解释需要对应实际代码，伪代码依赖反编译恢复质量，求解结论则受公式建模范围约束 [51, 56, 58]。后续研究需要明确每个结论依赖哪些地址、语句、控制关系和数据关系，并针对竞争性解释寻找反证。还应区分未发现修复、漏洞相关功能不适用和分析未完成，避免将工具失败或检索无结果直接转换为漏洞状态判断。

(四) 不同研究条件下，能力与失败机制的可比较性 代表性 PPT 研究分别采用内核、通用 C/C++ 项目、Java 字节码或 Android 应用，并在参考材料、定位支持与编译条件上具有差异 [4, 46–54]。论文中的单一准确率或 F1 值脱离实验设置后缺乏可比性。需要在统一条件下分别考察特征生成、目标定位、语义判定与完整流程，明确是否提供符号、候选函数和调试信息，并同时记录错误判定、无法判定、超时与工具异常。只有保留这些失败情形，才能进一步判断局限源于补丁语义不可区分、实现假设不满足，还是工程处理过程不完整，避免将所有失败混为模型或算法精度问题。

(五) 智能体分析效果、工具配置与成本之间的权衡 交互式分析扩大了系统可以探索的证据空间，也增加了工具调用、上下文维护和重复推理的成本。通用智

能体研究已经揭示接口设计和资源评价的重要性 [35, 61]，PPT 场景中”更大模型””更多工具”或”更长上下文”的效果仍需要受控实验验证。需要针对漏洞元数据、模型配置、工具能力与任务预算开展受控比较，研究如何优先使用低成本线索缩小范围，再针对关键证据缺口调用复杂分析，并同时评价正确性、完成率、时延和资源消耗。

综上，现有研究已经从补丁局部差异匹配发展到符号语义分析，并开始引入 LLM 进行跨表示定位与语义判别。然而，面向实际部署产物的可靠 PPT，仍需要贯通补丁理解、无充分提示的目标定位、演化修复识别和证据约束下的结论生成。对这些环节进行统一评估与失败机理分析，并探索程序分析和智能体推理之间的有效分工，是本课题研究内容三的主要切入点。

三 课题主要研究内容与预期目标

图 1 给出了本课题三个研究内容的总体概览。

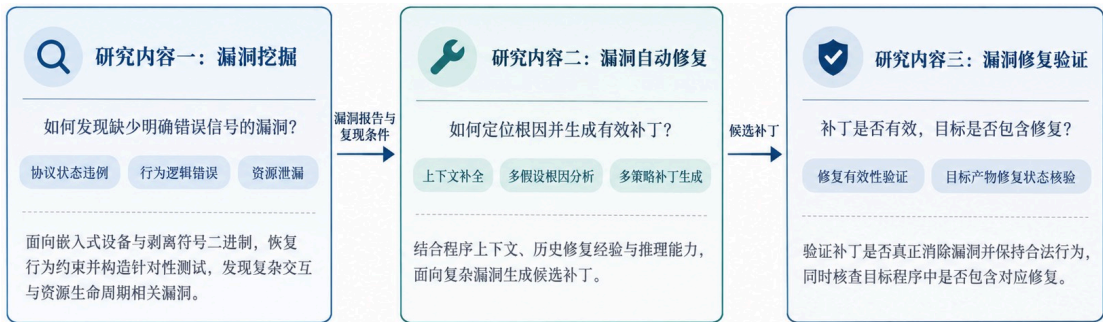


图 1 研究内容总览

3.1 研究目标

漏洞治理不仅需要发现潜在的安全缺陷，还需要针对漏洞形成有效修复，并确认修复满足相应安全要求、进入实际部署程序。仅发现漏洞或生成补丁，尚不足以确认目标系统中的相应风险已经消除。因此，本课题围绕“发现问题—生成修复—确认修复”的研究主线，探索大模型语义理解、智能体推理与程序分析相结合的方法，提升漏洞治理关键环节的自动化水平与结果可靠性。总体研究框架如图 2 所示。

在漏洞挖掘环节，本课题重点关注嵌入式设备与剥离符号二进制场景中的

协议状态违例、行为逻辑错误和资源泄漏等问题。此类问题的发现不仅需要触达相关执行路径，还需要判断程序行为是否违反协议、接口或资源生命周期要求。为此，拟研究受限信息条件下的行为约束恢复与针对性测试方法，将复杂交互中的状态变化与安全要求联系起来，提高对不易通过崩溃信号直接识别的漏洞的发现能力。

在漏洞修复环节，漏洞报告所指向的异常位置未必是实际需要修改的根因位置，而上下文缺失与过早收敛于单一解释，也可能限制修复策略的选择

在漏洞修复验证环节，补丁通过已知漏洞触发测试与功能测试，并不能充分说明漏洞根因已经消除[40]；同时，参考补丁的存在也不能直接证明实际部署的二进制包含对应修复[63]。为此，本课题拟从修复有效性与目标产物修复状态两个层面开展研究：一方面检查补丁是否满足漏洞相关安全要求并保持必要的合法行为，另一方面分析目标程序是否包含对应修复，提高对不完整修复、功能回归及修复缺失的识别能力。

基于上述目标，本课题形成**基于智能体的漏洞挖掘、基于智能体的漏洞自动修复、基于智能体的漏洞修复验证**三个研究内容。三者分别面向漏洞发现、候选补丁生成与修复结果确认，在任务目标上逐步衔接，共同为从漏洞识别到修复确认提供方法与技术支撑。

3.2 研究内容一：基于智能体的漏洞挖掘方法

主要研究内容：语义/逻辑漏洞（如认证绕过、状态机错误、配置驱动缺陷）不产生崩溃信号，长期缺乏高效的自动化挖掘手段，是IoT与嵌入式固件安全中的突出短板。其核心难点在于：此类漏洞的触发路径通常由内部状态、配置、时序与外设响应等**自因变量**共同决定，传统输入驱动的模糊测试难以在错误的维度上有效探索；同时，语义漏洞的判定依赖人工构造的策略与规约，oracle的缺失使自动化检测难以规模化。本研究内容旨在通过统一建模自因变量、自动构造语义 oracle、设计自因变量感知的智能体模糊测试，突破漏洞挖掘的自动化瓶颈，扩展可检测漏洞的类型与规模。

当前工作存在**自因变量难以统一建模、语义 oracle 难以自动构造**的挑战，因此，本文提出**自因变量感知的多智能体漏洞挖掘框架**。具体而言，首先综合静态分析与智能体推理，在固件二进制上识别各类自因变量，并统一表示为标注来源

应用层：漏洞攻防闭环

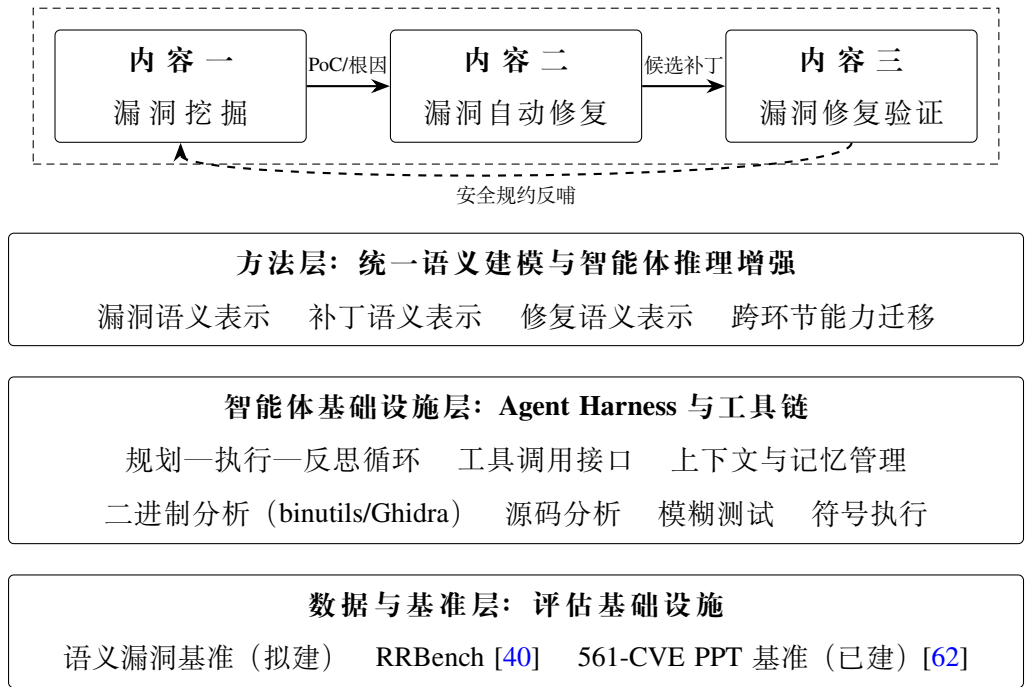


图 2 总体研究框架

类型、取值空间与影响路径的自因变量格，使其成为模糊测试的一等探索维度；其次，利用智能体从协议 RFC、芯片数据手册与配置说明等自然语言文档中自动提取安全不变式，并编译为可执行的运行时监测点与断言，解决 oracle 自动构造难题；最后，设计规划—执行—验证三智能体协同的模糊测试架构，由规划智能体根据覆盖率与不变式反馈规划探索策略，执行智能体在生成输入的同时主动翻转目标自因变量（如注入中断、修改 NVRAM、模拟外设响应），验证智能体检查不变式违背并报告候选漏洞，并配套 PoC 生成与根因定位能力。

预期目标：构建基于智能体的漏洞挖掘系统，在真实 IoT 固件与合成基准上验证有效性，相比现有最优模糊测试方法在漏洞发现数量上提升 ≥ 5 倍，自因变量覆盖率 $\geq 70\%$ ，安全 oracle 自动构造准确率 $\geq 80\%$ ，在认证绕过、状态机错误与配置驱动缺陷三类典型漏洞上各发现若干真实漏洞并获得 CVE 编号，形成可复用的固件语义漏洞基准与工具原型。

3.3 研究内容二：基于智能体的漏洞自动修复方法

主要研究内容：漏洞自动修复是连接漏洞发现与漏洞消除的关键环节，其目标是自动定位漏洞根因、合成补丁并验证正确性。以 RRBench 为代表的大规

模评估揭示了一个关键问题：现有智能体生成的“公开合理”补丁中，有相当比例（23.4%）在私有安全性质检查下失败，且存在多个案例无法被任何系统正确修复 [40]。其根本原因在于：修复环节缺乏对“修复意图”的语义判定能力，智能体容易生成仅抑制特定 PoC、却未消除漏洞根因的过拟合补丁。值得注意的是，这一问题与补丁存在性测试中的“补丁语义判定”在本质上是对偶的——前者回答“生成的补丁是否构成有效修复”，后者回答“目标二进制是否包含修复”——两者均需对漏洞语义与修复语义进行深层建模。本研究内容旨在构建规约引导的可信自动修复框架，生成高质量候选补丁，并与研究内容三的修复验证方法形成“生成—验证”协同。

当前工作存在**部分规约下修复正确性难以验证、过拟合与回归难以系统抑制**的挑战，因此，本文提出**规约引导的智能体漏洞修复框架**。具体而言，首先依托统一的漏洞语义五元组表示（与研究内容一、三共享），将漏洞报告与 PoC 转化为结构化的根因语义表示；其次，借鉴 RRBench 私有 Oracle 的设计思想但将其自动化，利用智能体基于漏洞语义与代码上下文自动提取补丁必须满足的安全性质集合，既指导补丁生成又作为最终验证依据；再次，修复智能体在根因语义与安全规约的双重引导下进行多轮补丁生成与迭代，通过编译、公开测试与静态检查反馈持续修正候选补丁；最后，候选补丁交由研究内容三的修复验证环节进行 PPT 式的修复正确性判定，二者构成“生成—验证”对偶闭环，系统性抑制过拟合补丁与合法功能回归，实现部分规约下可信的自动修复。

预期目标：构建基于智能体的漏洞自动修复系统，在 RRBench 基准上将正确修复率从当前最优 69.6% 提升至 $\geq 80\%$ ，私有验证拒绝率降低 $\geq 50\%$ ，在现有任何系统均无法正确修复的 14 个案例上成功修复 ≥ 4 个，形成可复用的安全规约自动提取工具链，并将方法论扩展至 C/C++ 等其他语言生态的修复基准。

3.4 研究内容三：基于智能体的漏洞修复验证方法

主要研究内容：补丁存在性测试（Patch Presence Testing, PPT）是连接漏洞披露与实际防护的关键环节，在软件供应链安全审计中具有不可替代的作用；同时，修复验证也是漏洞自动修复闭环的收官环节——自动合成的补丁是否真正消除漏洞根因，最终需要同样的补丁语义判定能力来回答。本人前期实证研究表明，前沿编码智能体已在端到端补丁存在性测试上达到与专用 SOTA 系统相当

的性能 [63]，但其失败高度集中于两个环节：剥离二进制中的补丁定位，以及对后续版本中演化补丁的识别。前者源于源码级补丁意图与二进制级指令序列之间存在跨抽象层次的语义鸿沟，后者源于现有方法过度依赖补丁的形式特征而非修复意图。本研究内容旨在通过引入统一的漏洞语义表示，增强智能体在两个瓶颈环节上的推理能力，推动端到端修复状态验证从“可用”走向“可信、可扩展”，并将补丁语义判定能力对偶迁移至自动修复的补丁正确性验证。

当前工作存在**跨抽象层次补丁语义对齐困难、修复意图难以形式化**的挑战，因此，本文提出**语义先验引导的智能体修复验证增强框架**。具体而言，离线阶段利用智能体从 CVE 描述、补丁 diff 与源码上下文中提取“触发条件—危险操作—干预点—修复意图—上下文”的漏洞语义五元组，作为统一语义先验；在线阶段采用“粗—细”两阶段补丁定位，先用启发式规则快速缩小候选函数集，再以五元组为约束引导智能体在反汇编切片中进行置信度排序；针对演化补丁，提出基于修复意图的语义等价判定方法，不再追求形式上的补丁匹配，而是验证目标二进制中的干预是否实现了与原始补丁相同的安全语义。该框架在保持推理成本可控的前提下，系统性提升补丁定位准确率与演化补丁识别能力。

预期目标：构建基于智能体的端到端漏洞修复验证系统，在 561-CVE 基准上实现 $DSR \geq 0.98$ ，在演化补丁子集上 DSR 提升 $\geq 15\%$ ，在 Ubuntu 真实发行版包上保持 $DSR \geq 0.95$ ，单例推理成本相比前沿配置增加不超过 10%，形成可支撑大规模软件供应链审计与修复正确性判定的工具原型。

3.5 预期目标与创新点

（待撰写：分点列出理论创新、方法创新与系统/工具成果，以及预期论文与专利产出。）

四 拟采用的研究方法、技术路线、实验方案及其可行性分析

4.1 基于智能体的漏洞挖掘方法

拟解决的关键问题：语义/逻辑漏洞（认证绕过、状态机错误、配置驱动缺陷）自动化挖掘的两大瓶颈——**自因变量难以统一建模与语义 oracle 难以自动构造**。自因变量——那些不受外部输入直接控制、由内部状态、配置、时序与外设决定

的变量——在文献中以 state-driven、config-driven、timing-driven、peripheral-driven 等不同形态各自为战，缺乏统一抽象；同时，语义漏洞的判定依赖人工构造的策略与规约，Firmalice 的认证绕过 oracle 使用十年仍是孤例 [16]，oracle 的缺失使自动化检测难以规模化。

技术方案：针对上述问题，本文提出**自因变量感知的多智能体漏洞挖掘框架**，整体技术路线如图 3 所示，包含四个核心模块：

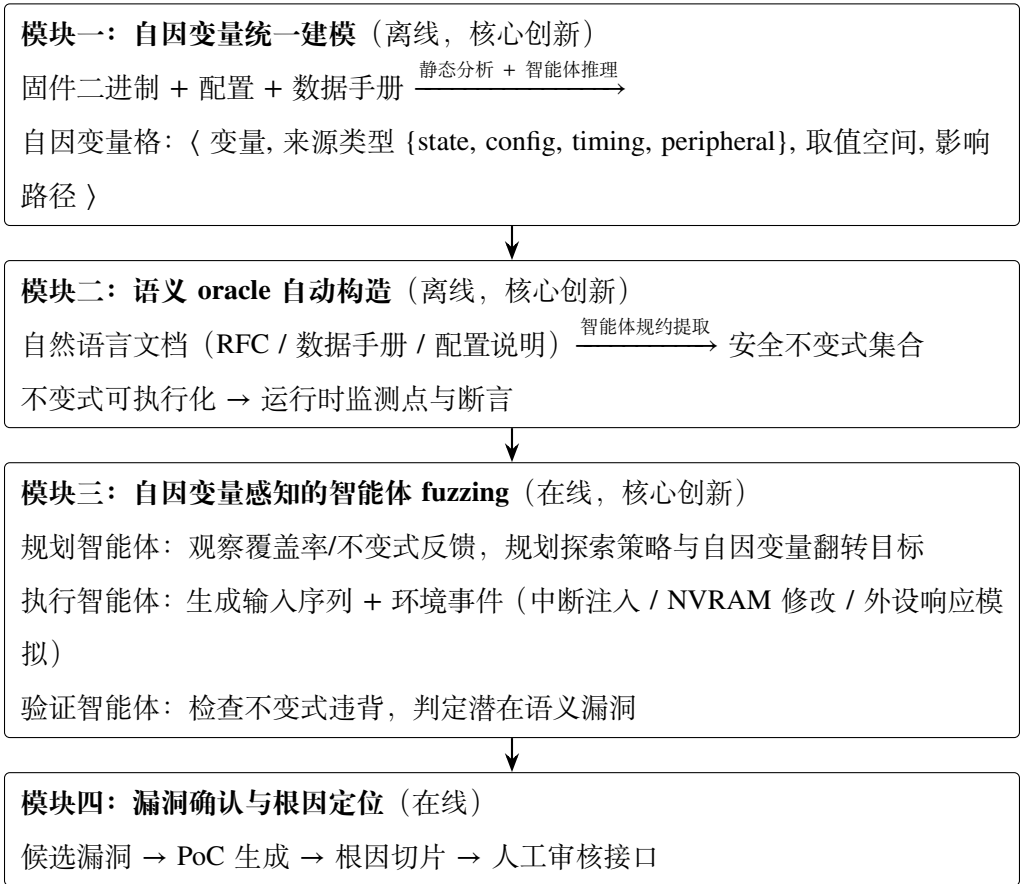


图 3 基于智能体的漏洞挖掘技术路线

- **模块一：自因变量统一建模。**综合静态分析与智能体推理，在固件二进制上识别各类自因变量，并统一表示为标注来源类型、取值空间与影响路径的自因变量格，使后续 fuzzing 能够将自因变量作为一等探索维度。
- **模块二：语义 oracle 自动构造。**利用智能体从自然语言文档（协议 RFC、芯片数据手册、配置说明）中自动提取安全不变式，并将其编译为可执行的运行时监测点与断言，解决漏洞挖掘中“oracle 靠人工”的长期瓶颈。

- **模块三：自因变量感知的智能体 fuzzing**。采用规划—执行—验证三智能体协同架构：规划智能体根据覆盖率与不变式反馈规划探索策略；执行智能体在生成输入的同时，主动翻转目标自因变量；验证智能体检查不变式违背。该模块突破传统 fuzzing 仅在输入空间探索的局限。
- **模块四：漏洞确认与根因定位**。对验证智能体报告的候选漏洞，自动生成 PoC，通过动态切片定位根因，并提供人工审核接口。

可行性分析：（1）理论可行：自因变量概念已在 SyzTrust [64]、Fuzzware [8] 等工作中以不同形态出现，本方案是对这些分散实践的统一与提升；（2）技术可行：多智能体协同架构已在 FirmAgent [22] 中得到初步验证，LLM 的规约理解能力已在 ChatAFL [21] 等工作中展现，“语义检索 + 可验证推理”的 agentic 范式已在通用二进制分析任务上被证明有效 [7]；（3）工程可行：课题组在固件分析、智能体框架与 fuzzing 工具链上有完整积累，RRBench 的“角色分离智能体”设计模式可直接复用 [40]；（4）数据可行：可通过公开固件数据集与 FreeRTOS、Zephyr 等开源 IoT 项目获取评估对象，并可人工植入语义漏洞构建合成基准。

实验方案：基准包括 10–15 个含已知语义漏洞的真实 IoT 固件（含认证绕过、状态机错误等）与参考 LAVA-M 设计的 50+ 合成植入漏洞；对比方法包括 FIRM-AFL [65]、Fuzzware [8]、FirmAgent [22]、ChatAFL [21]、AFLNet [17]；评估指标包括语义漏洞发现数量、首次发现时间、假阳性率、自因变量覆盖率、oracle 自动构造准确率；消融实验分别验证自因变量建模、自动 oracle 与多智能体协同的独立贡献。预期语义漏洞发现数量相比最优基线提升 ≥ 5 倍，自因变量覆盖率 $\geq 70\%$ ，oracle 自动构造准确率 $\geq 80\%$ 。

4.2 基于智能体的漏洞自动修复方法

拟解决的关键问题：智能体漏洞自动修复的两大瓶颈——**部分规约下修复正确性难以验证与过拟合与回归难以系统抑制**。RRBench 大规模评估表明，现有智能体生成的“公开合理”补丁中 23.4% 在私有安全性质检查下失败，且 14 个案例未被任何系统正确修复 [40]。其根本原因在于：修复环节缺乏对“修复意图”的语义判定能力，智能体容易生成仅抑制特定 PoC、却未消除漏洞根因的过拟合补丁，或在修复漏洞的同时破坏合法功能。

技术方案：针对上述问题，本文提出**修复—验证对偶的智能体漏洞修复框架**，核心思想是将 PPT 的”补丁语义判定”能力对偶迁移为”修复正确性验证”，整体技术路线如图 4 所示，包含四个核心模块：



图 4 基于智能体的漏洞自动修复技术路线

- **模块一：漏洞根因语义理解。**依托统一的漏洞语义五元组表示（与研究内容三共享），将漏洞报告与 PoC 转化为结构化的根因语义表示，为后续修复提供深层理解基础。
- **模块二：安全性质规约自动提取。**这是连接 PPT 与修复的关键创新。借鉴 RRBench 私有 Oracle 的设计思想，但将其从”人工构造”扩展为”智能体自动生成”：智能体基于漏洞语义与代码上下文，自动提取补丁必须满足的安全性质集合。这些规约既可指导补丁生成，也可作为最终验证依据。
- **模块三：规约引导的补丁生成。**修复智能体在根因语义与安全规约的双重引导下生成候选补丁，并通过编译、公开测试与静态检查进行多轮迭代。
- **模块四：PPT 式修复验证。**这是本研究内容的核心创新，将 PPT 能力对偶

迁移到修复验证：通过**补丁语义判定**（验证候选补丁是否真正实现应有不变式）、**差分测试**（自动构造与 PoC 语义相近但不同的输入以检测过拟合）、**回归检查**（验证合法功能保持）三重机制，系统性抑制过拟合补丁与合法功能回归。

可行性分析：（1）直接基于 RRBench：基准、评估框架、五款基线 agent 数据均已就绪，可快速开展对比实验，与课题组前期工作延续性强 [40]；（2）PPT 能力可迁移：研究内容三的”修复意图判定”与本内容的补丁验证需求本质同源，技术路线已形成闭环，风险分散；（3）规约自动提取有先例：ChatAFL 的协议规约提取、Firmalice 的特权模型等已证明 LLM 具备规约理解能力 [16, 21]；（4）工程基础成熟：SWE-agent [35]、OpenHands [66] 等开源框架提供了成熟的修复智能体底座。

实验方案：基准采用 RRBench 全部 115 个案例，后续扩展至 SEC-bench [37]、CVE-Bench [67] 子集；对比方法包括 SWE-agent [35]、OpenHands [66]、RepairAgent [68]、RustAgent、RustForger（即 RRBench 原五款）；评估指标包括 Correct Repair Rate（通过公开 + 私有验证）、Plausible-but-Private-Rejected Rate、平均迭代轮数、每例成本；消融实验分别验证安全规约引导、PPT 式验证与差分测试的独立贡献。预期 Correct Repair Rate $\geq 80\%$ ，私有拒绝率降低 $\geq 50\%$ ，在 14 个”无人能修”案例上成功修复 ≥ 4 个。

4.3 基于智能体的漏洞修复验证方法

拟解决的关键问题：智能体在端到端漏洞修复验证（以补丁存在性测试为核心任务）中的两大瓶颈——**剥离二进制中的补丁定位与演化补丁识别**。前期实证研究表明，前者占弱模型配置独有失败的 95.2%，后者是鲁棒性测试的主要失分点 [63]。两者本质上都源于跨抽象层次补丁语义对齐困难、修复意图难以形式化：源码级补丁意图与二进制级指令序列之间存在语义鸿沟，且现有方法过度依赖补丁的形式特征而非修复意图。

技术方案：针对上述问题，本文提出**语义先验引导的智能体修复验证增强框架**，整体技术路线如图 5 所示，包含三个核心模块：

- **模块一：漏洞语义先验提取。**离线阶段利用智能体从 CVE 描述、补丁 diff



图 5 基于智能体的漏洞修复验证技术路线

与源码上下文中提取漏洞语义五元组，作为统一语义先验，替代现有方法仅依赖原始补丁 diff 的浅层表示。

- **模块二：语义引导的补丁定位。**在线阶段采用“粗—细”两阶段定位：先用传统启发式（字符串、常量、导入表）快速缩小候选函数集，再将五元组作为约束注入智能体推理，引导其在候选函数中进行语义切片与置信度排序，输出 Top-K 补丁候选区域。
- **模块三：演化补丁的语义等价判定。**针对同一漏洞可能存在多种实现形式的修复（重构、拆分、合并等），不再追求“形式上的补丁匹配”，而是让智能体判断目标二进制中的干预点是否实现了与原始补丁相同的修复意图，并通过语义不变式进行交叉验证。

可行性分析：（1）数据基础充分：已构建 561 CVE 高保真基准与 12 组智能体配置的运行日志 [62, 63]，包含大量补丁定位与演化补丁的成功/失败案例，可直接用于方法设计与消融验证；（2）技术路线有实证支撑：前期工作已证明“丰富补丁元数据提升 DSR”与“补丁定位是主要瓶颈”两个关键结论，本方案是这两个结论的直接延伸，技术风险低；（3）工具链成熟：智能体框架（Codex / MiniSWEAgent）、二进制分析工具（binutils / Ghidra）与评估基准均已就绪，可

快速进入实验迭代。

实验方案：基准沿用前期 561-CVE Source-Built Benchmark + Patch Evolution Set + Ubuntu Packages Set；对比方法包括 Codex 原始配置、Lares [59]、PPTSP [60]、PS³ [50]、BinXray [46]；评估指标包括 DSR、Accuracy、补丁定位准确率、每例 token 成本与时延；消融实验分别验证三个模块的独立贡献。预期全配置 DSR ≥ 0.98 ，演化补丁子集 DSR 提升 $\geq 15\%$ ，成本增加 $\leq 10\%$ ；进一步将验证能力对偶迁移至修复场景，在 RRBench 生成补丁上验证修复正确性判定的有效性。

4.4 实验方案设计

4.4.1 整体评估策略

三个研究内容的实验遵循统一策略：

1. **基准优先：**每个内容均基于已有或新建的高质量基准，确保可复现与可比较。
2. **多维度评估：**从效果（准确率/发现数/修复率）、效率（时间/token 成本）、鲁棒性（跨配置/跨场景）三个维度综合评估。
3. **消融驱动：**每个核心模块均设计消融实验，验证其独立贡献。
4. **跨内容联动：**设计端到端实验，验证三个内容构成的闭环是否产生协同增益。

4.4.2 端到端闭环验证实验

为验证三个研究内容构成的攻防闭环的有效性，设计如下端到端实验：

1. **输入：**一批含已知漏洞的 IoT 固件镜像（部分为真实 CVE，部分为人工植入）。
2. **流程：**内容一挖掘未知语义漏洞，产出根因与 PoC；内容二对确认的漏洞进行自动修复，产出候选补丁；内容三对修复后固件进行 PPT 式修复验证，判定补丁是否真正消除漏洞根因；同时内容三对固件中已知 CVE 的修复状态进行补丁存在性核查，覆盖供应链审计场景。

3. **评估：**闭环整体准确率、各环节能力复用带来的效率提升、相比独立运行的性能差异。

4.4.3 基准建设计划

表 1 基准建设计划

基准	状态	规模	用途
561-CVE PPT 基准	已建成	561 CVE / 10 项目	内容三
语义漏洞固件基准	拟建设	10-15 真实固件 + 50 合成案例	内容一
RRBench	已建成 (课题组)	115 案例 / 95 仓库	内容二
端到端闭环基准	拟整合	复用上述 + 新增修复验证	闭环验证

4.4.4 风险与应对

表 2 风险与应对策略

风险	影响	应对策略
语义 oracle 自动构造准确率不足	内容一假阳性率高	引入人工审核接口；采用多智能体交叉验证
自因变量建模在复杂固件上扩展性差	内容一覆盖率不达预期	从中小型固件起步逐步扩展；结合动态污点辅助识别
修复验证的私有规约生成不完整	内容二私有拒绝率下降有限	结合官方补丁对比学习；引入形式化方法辅助规约补全
智能体推理成本过高	方法难以规模化	分级模型策略（轻量粗筛 + 重量精判）；缓存中间语义表示

五 已有科研基础与所需的科研条件

5.1 已有科研基础

本人已经针对本文涉及的相关研究进行了充分调研和分析，积累了相应理论和实践基础。

针对研究内容一（基于智能体的漏洞挖掘），本人正在开展相关调研与原型设计工作。

针对研究内容二（基于智能体的漏洞自动修复），本人深度参与课题组 RRBench 基准的讨论与分析工作，熟悉其两阶段评估设计与五款基线智能体的运行数据 [40]，为后续修复研究奠定了直接基础。

针对研究内容三（基于智能体的漏洞修复验证），本人已完成两项相互衔接的实证研究：（1）构建了包含 561 个 CVE、覆盖 10 个广泛使用开源项目的高保真 PPT 基准，对五款代表性工具进行了系统评估，揭示了“高准确率、低决策成功率”的普遍现象，并提出改进策略取得显著提升，成果发表于 ISSTA 2026 [62]；（2）系统评估了编码智能体在端到端 PPT 任务中的能力边界与配置空间，识别出补丁定位与演化补丁识别两大瓶颈，成果已完成论文撰写 [63]。

（待补充：项目参与情况、其他安全分析实践成果。）

5.2 所需科研条件

（待撰写：计算资源（GPU 集群与 LLM API 预算）、数据资源（固件语料、CVE 数据库）、团队与项目支撑。）

六 研究工作计划与进度安排

1. **2026.09 – 2027.03**：完成研究内容一。构建自因变量统一建模与语义 oracle 自动构造方法，实现多智能体漏洞挖掘原型，建设固件漏洞基准，撰写并发表研究内容一论文。
2. **2027.03 – 2027.12**：完成研究内容二。实现规约引导的智能体漏洞修复框架，在 RRBench 上完成系统评估，撰写并发表研究内容二论文。
3. **2028.01 – 2028.10**：完成研究内容三。设计并实现语义先验引导的智能体修复验证（PPT 增强）框架，在 561-CVE 基准上完成对比与消融实验，撰写并发表研究内容三论文。
4. **2028.10 – 2029.06**：完成三个研究内容的端到端闭环集成与验证，整理成果，撰写博士学位论文，准备答辩。

(注：以上时间节点为初步规划，可按实际学制与导师意见调整。)

参考文献

- [1] Center N C S R. Vulnerability — glossary [EB/OL]. 2026. <https://csrc.nist.gov/glossary/term/vulnerability>.
- [2] Householder A D, Coughlin E, Kennedy-McHugh K, et al. The CERT guide to coordinated vulnerability disclosure: CMU/SEI-2017-SR-015 [R]. CERT Division, Software Engineering Institute, Carnegie Mellon University, 2017.
- [3] Hu Y, Li Z, Shu K, et al. Sok: Automated vulnerability repair: Methods, tools, and assessments [C]//34th USENIX Security Symposium. 2025: 4421-4440.
- [4] Zhang H, Qian Z. Precise and accurate patch presence test for binaries [C]//27th USENIX Security Symposium. 2018: 887-902.
- [5] Google Project Zero and DeepMind. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code [EB/OL]. 2024. <https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html>.
- [6] Yu Z, Guo Z, Wu Y, et al. PatchAgent: A practical program repair agent mimicking human expertise [C]//34th USENIX Security Symposium. 2025: 4381-4400.
- [7] Liu Y, Liu Z, Yu Z, et al. Towards generality: Task-adaptive binary analysis via semantic retrieval and verifiable reasoning [C]//35th USENIX Security Symposium. 2026.
- [8] Scharnowski T, Bars N, Schloegel M, et al. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing [C]//31st USENIX Security Symposium. 2022: 1239-1256.
- [9] Barr E T, Harman M, McMinn P, et al. The oracle problem in software testing: A survey [J/OL]. IEEE Transactions on Software Engineering, 2015, 41(5): 507-525. DOI: [10.1109/TSE.2014.2370955](https://doi.org/10.1109/TSE.2014.2370955).
- [10] Li Z, Wang N, Zou D, et al. On the effectiveness of function-level vulnerability detectors for inter-procedural vulnerabilities [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639218](https://doi.org/10.1145/3597503.3639218).
- [11] Nong Y, Yang H, Cheng L, et al. APPATCH: Automated adaptive prompting large language models for real-world software vulnerability patching [C]//34th USENIX Security Symposium. 2025: 4481-4500.
- [12] Smith E K, Barr E T, Le Goues C, et al. Is the cure worse than the disease? overfitting in automated program repair [C/OL]//Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015: 532-543. DOI: [10.1145/2786805.2786825](https://doi.org/10.1145/2786805.2786825).
- [13] Manès V J M, Han H, Han C, et al. The art, science, and engineering of fuzzing: A survey

- [J/OL]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331. DOI: [10.1109/TSE.2019.2946563](https://doi.org/10.1109/TSE.2019.2946563).
- [14] Serebryany K, Bruening D, Potapenko A, et al. AddressSanitizer: A fast address sanity checker [C]//2012 USENIX Annual Technical Conference. 2012: 309-318.
- [15] Mozilla. Frontier-model-assisted vulnerability discovery in Firefox [Z]. 2026.
- [16] Shoshitaishvili Y, Wang R, Hauser C, et al. Fomalice: Automatic detection of authentication bypass vulnerabilities in binary firmware [C]//Network and Distributed System Security Symposium (NDSS). 2015.
- [17] Pham V T, Böhme M, Roychoudhury A. AFLNet: A greybox fuzzer for network protocols [C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 460-465.
- [18] Natella R. StateAFL: Greybox fuzzing for stateful network servers [C]//2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2022.
- [19] Meng R, Mirchev M, Böhme M, et al. Linear-time temporal logic guided greybox fuzzing [C/OL]//Proceedings of the 44th International Conference on Software Engineering. 2022: 765-777. DOI: [10.1145/3510003.3510082](https://doi.org/10.1145/3510003.3510082).
- [20] Xia C S, Palma F, Jia M L, et al. Fuzz4All: Universal fuzzing with large language models [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024. DOI: [10.1145/3597503.3639121](https://doi.org/10.1145/3597503.3639121).
- [21] Meng R, Mirchev M, Böhme M, et al. Large language model guided protocol fuzzing [C]//Network and Distributed System Security Symposium (NDSS). 2024.
- [22] Ji J, Zhang C, Gan S, et al. FirmAgent: Leveraging fuzzing to assist LLM agents with IoT firmware vulnerability discovery [C/OL]//Network and Distributed System Security Symposium (NDSS). 2026. DOI: [10.14722/ndss.2026.231943](https://doi.org/10.14722/ndss.2026.231943).
- [23] Yang T, Qin Y, Zhang L, et al. BSFuzzer: Context-aware semantic fuzzing for BLE logic flaw detection [C]//Network and Distributed System Security Symposium (NDSS). 2026.
- [24] Song X, Pei L, Wu J, et al. ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification [C]//Network and Distributed System Security Symposium (NDSS). 2026.
- [25] Wang R, Li T, Li J. TyPatch: Transforming patches into tpestate rules for kernel bug detection [Z]. 2026.
- [26] Yang R, He Y, Xu Y, et al. LLM-based invariant testing for software functional bugs [C]//2026 IEEE International Symposium on Software Reliability Engineering (ISSRE). 2026.
- [27] Wang C, Liu J, Peng X, et al. Boosting static resource leak detection via LLM-based resource-

- oriented intention inference [C]//Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. 2025.
- [28] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: Securing the foundations of the Rust programming language [J/OL]. Proceedings of the ACM on Programming Languages, 2018, 2 (POPL): 1-34. DOI: [10.1145/3158154](https://doi.org/10.1145/3158154).
- [29] Qin B, Chen Y, Yu Z, et al. Understanding memory and thread safety practices and issues in real-world Rust programs [C/OL]//Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation. 2020: 763-779. DOI: [10.1145/3385412.3386036](https://doi.org/10.1145/3385412.3386036).
- [30] Le Goues C, Nguyen T, Forrest S, et al. GenProg: A generic method for automatic software repair [J]. IEEE Transactions on Software Engineering, 2012, 38(1): 54-72.
- [31] Nguyen H D T, Qi D, Roychoudhury A, et al. SemFix: Program repair via semantic analysis [C/OL]//2013 35th International Conference on Software Engineering. 2013: 772-781. DOI: [10.1109/ICSE.2013.6606623](https://doi.org/10.1109/ICSE.2013.6606623).
- [32] Gao X, Wang B, Duck G J, et al. Beyond tests: Program vulnerability repair via crash constraint extraction [J/OL]. ACM Transactions on Software Engineering and Methodology, 2021, 30 (2): 1-27. DOI: [10.1145/3418461](https://doi.org/10.1145/3418461).
- [33] Fu M, Tantithamthavorn C, Le T, et al. VulRepair: A T5-based automated software vulnerability repair [C/OL]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 935-947. DOI: [10.1145/3540250.3549098](https://doi.org/10.1145/3540250.3549098).
- [34] Pearce H, Tan B, Ahmad B, et al. Examining zero-shot vulnerability repair with large language models [C/OL]//2023 IEEE Symposium on Security and Privacy. 2023: 2339-2356. DOI: [10.1109/SP46215.2023.10179420](https://doi.org/10.1109/SP46215.2023.10179420).
- [35] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C]//Advances in Neural Information Processing Systems: volume 37. 2024: 50528-50652.
- [36] Bui Q C, Scandariato R, Díaz Ferreyra N E. Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques [C/OL]//Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 464-468. DOI: [10.1145/3524842.3528482](https://doi.org/10.1145/3524842.3528482).
- [37] Lee H, Zhang Z, Lu H, et al. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks [C]//Advances in Neural Information Processing Systems: volume 38. 2025.

- [38] Yu Z, Shi W, Sun X, et al. Patch validation in automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2603.06858](https://doi.org/10.48550/arXiv.2603.06858).
- [39] Shen C, Li J, Mahajan A, et al. PatchBench: Evaluating AI agents for vulnerability patching [EB/OL]. 2026. DOI: [10.48550/arXiv.2609.04075](https://doi.org/10.48550/arXiv.2609.04075).
- [40] Anonymous. RRBench: A repository-level benchmark for agentic repair of real-world Rust vulnerabilities [Z]. 2026.
- [41] Wang H, Basque Z L, Hu J, et al. Root-cause-driven automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2605.04251](https://doi.org/10.48550/arXiv.2605.04251).
- [42] Li F, Paxson V. A large-scale empirical study of security patches [C/OL]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 2017: 2201-2215. DOI: [10.1145/3133956.3134072](https://doi.org/10.1145/3133956.3134072).
- [43] Machiry A, Redini N, Camellini E, et al. SPIDER: Enabling fast patch propagation in related software repositories [C/OL]//2020 IEEE Symposium on Security and Privacy. 2020: 1562-1579. DOI: [10.1109/SP40000.2020.00038](https://doi.org/10.1109/SP40000.2020.00038).
- [44] Zhang Z, Zhang H, Qian Z, et al. An investigation of the Android kernel patch ecosystem [C]//30th USENIX Security Symposium. 2021: 3649-3666.
- [45] Red Hat. Security backporting practice [EB/OL]. 2026. <https://access.redhat.com/security/updates/backporting>.
- [46] Xu Y, Xu Z, Chen B, et al. Patch based vulnerability matching for binary programs [C/OL]//Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2020: 376-387. DOI: [10.1145/3395363.3397361](https://doi.org/10.1145/3395363.3397361).
- [47] Xu X, Zheng Q, Yan Z, et al. PatchDiscovery: Patch presence test for identifying binary vulnerabilities based on key basic blocks [J/OL]. IEEE Transactions on Software Engineering, 2023, 49(12): 5279-5294. DOI: [10.1109/TSE.2023.3332732](https://doi.org/10.1109/TSE.2023.3332732).
- [48] Jiang Z, Zhang Y, Xu J, et al. PDiff: Semantic-based patch presence testing for downstream kernels [C/OL]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. 2020: 1149-1163. DOI: [10.1145/3372297.3417240](https://doi.org/10.1145/3372297.3417240).
- [49] Yang S, Xu Z, Xiao Y, et al. Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis [J/OL]. ACM Transactions on Software Engineering and Methodology, 2023, 32(6): 1-29. DOI: [10.1145/3604608](https://doi.org/10.1145/3604608).
- [50] Zhan Q, Hu X, Li Z, et al. PS³: Precise patch presence test based on semantic symbolic signature [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639134](https://doi.org/10.1145/3597503.3639134).
- [51] Zhan Q, Hu X, Xia X, et al. REACT: IR-level patch presence test for binary [C/OL]//

- Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 381-392. DOI: [10.1145/3691620.3695012](https://doi.org/10.1145/3691620.3695012).
- [52] Dai J, Zhang Y, Jiang Z, et al. BScout: Direct whole patch presence test for Java executables [C]//29th USENIX Security Symposium. 2020: 1147-1164.
- [53] Pan Z, Hu X, Xia X, et al. PPT4J: Patch presence test for Java binaries [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639231](https://doi.org/10.1145/3597503.3639231).
- [54] Xie Z, Wen M, Jia H, et al. Precise and efficient patch presence test for Android applications against code obfuscation [C/OL]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 347-359. DOI: [10.1145/3597926.3598061](https://doi.org/10.1145/3597926.3598061).
- [55] Xie Z, Wen M, Wei Z, et al. Unveiling the characteristics and impact of security patch evolution [C/OL]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 1094-1106. DOI: [10.1145/3691620.3695488](https://doi.org/10.1145/3691620.3695488).
- [56] Hu P, Liang R, Chen K. DeGPT: Optimizing decompiler output with LLM [C/OL]//Network and Distributed System Security Symposium (NDSS). 2024. DOI: [10.14722/ndss.2024.24401](https://doi.org/10.14722/ndss.2024.24401).
- [57] Tan H, Luo Q, Li J, et al. LLM4Decompile: Decompile binary code with large language models [C/OL]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 3473-3487. DOI: [10.18653/v1/2024.emnlp-main.203](https://doi.org/10.18653/v1/2024.emnlp-main.203).
- [58] Xie D, Zhang Z, Jiang N, et al. ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries [C/OL]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. 2024: 4554-4568. DOI: [10.1145/3658644.3670340](https://doi.org/10.1145/3658644.3670340).
- [59] Li S, Zheng Y, Li H, et al. Lares: LLM-driven code slice semantic search for patch presence testing [C/OL]//2025 40th IEEE/ACM International Conference on Automated Software Engineering. 2025: 919-931. DOI: [10.1109/ASE63991.2025.00081](https://doi.org/10.1109/ASE63991.2025.00081).
- [60] Xu C, Luo S, Duan X, et al. PPTSP: Patch presence test via semantic normalization and key path extraction [J/OL]. Software Quality Journal, 2026, 34(2). DOI: [10.1007/s11219-026-09760-9](https://doi.org/10.1007/s11219-026-09760-9).
- [61] Kapoor S, Stroebl B, Siegel Z S, et al. AI agents that matter [EB/OL]. 2024. DOI: [10.48550/arXiv.2407.01502](https://doi.org/10.48550/arXiv.2407.01502).
- [62] Zhang X, Zheng Y, Luo W, et al. A comprehensive empirical analysis of patch presence testing: Capabilities, limitations, and paths forward [J/OL]. Proceedings of the ACM on Software Engineering, 2026, 3(ISSTA): ISSTA053. DOI: [10.1145/3832144](https://doi.org/10.1145/3832144).
- [63] Zhang X, et al. Understanding coding agents for patch presence testing [Z]. 2026.
- [64] Wang D, et al. SyzTrust: State-aware fuzzing on trusted OS designed for IoT devices [M]//2024 IEEE Symposium on Security and Privacy. 2024.
- [65] Zheng Y, Davanian A, Yin H, et al. FIRM-AFL: High-throughput greybox fuzzing of IoT

- firmware via augmented process emulation [C]//28th USENIX Security Symposium. 2019: 1099-1114.
- [66] Wang X, Li B, Song Y, et al. OpenHands: An open platform for AI software developers as generalist agents [EB/OL]. 2024. DOI: [10.48550/arXiv.2407.16741](https://doi.org/10.48550/arXiv.2407.16741).
- [67] Anonymous. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities [Z]. 2024.
- [68] Bouzenia I, Devanbu P, Pradel M. RepairAgent: An autonomous, LLM-based agent for program repair [C/OL]//2025 IEEE/ACM 47th International Conference on Software Engineering. 2025: 2188-2200. DOI: [10.1109/ICSE55347.2025.00157](https://doi.org/10.1109/ICSE55347.2025.00157).